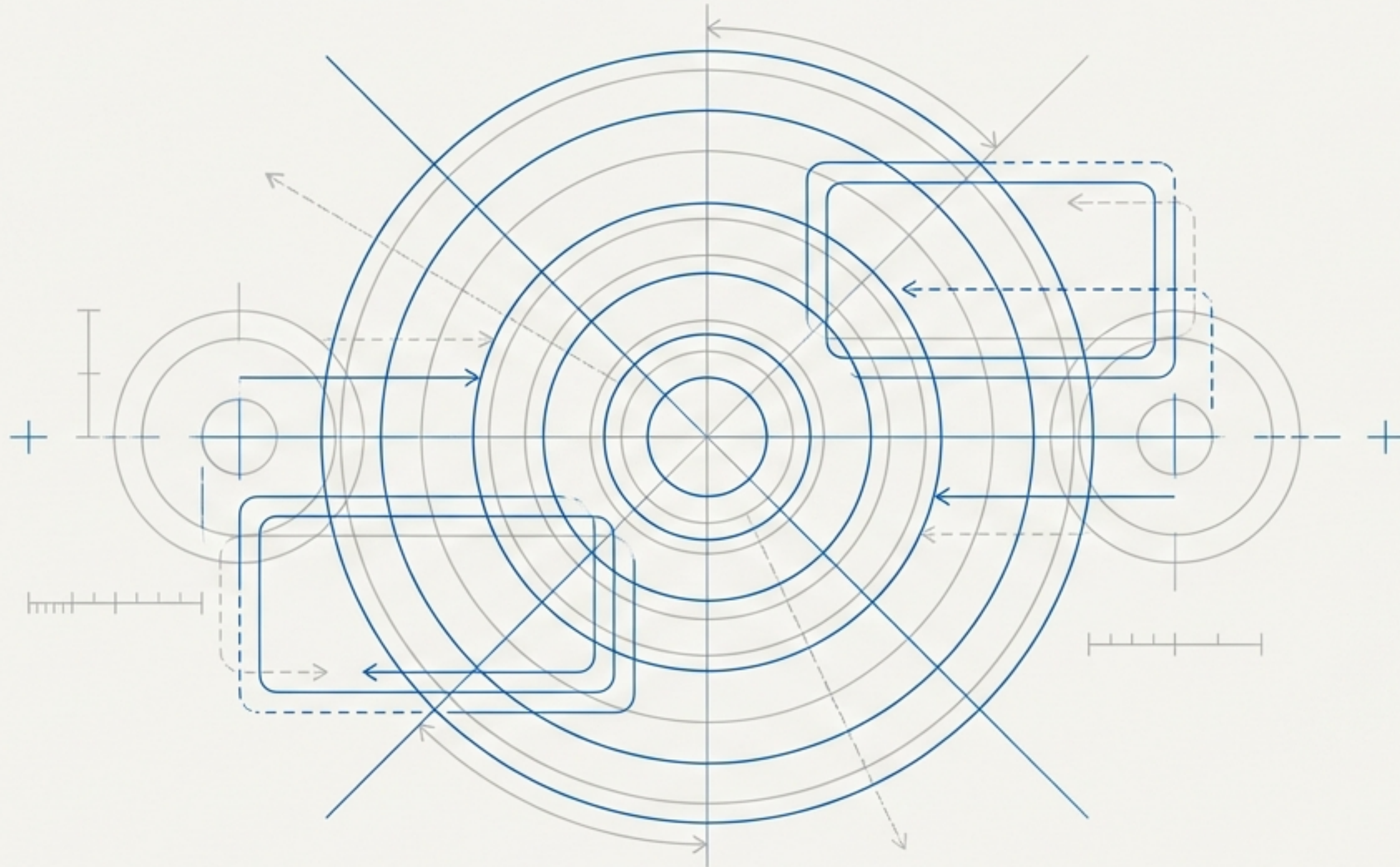
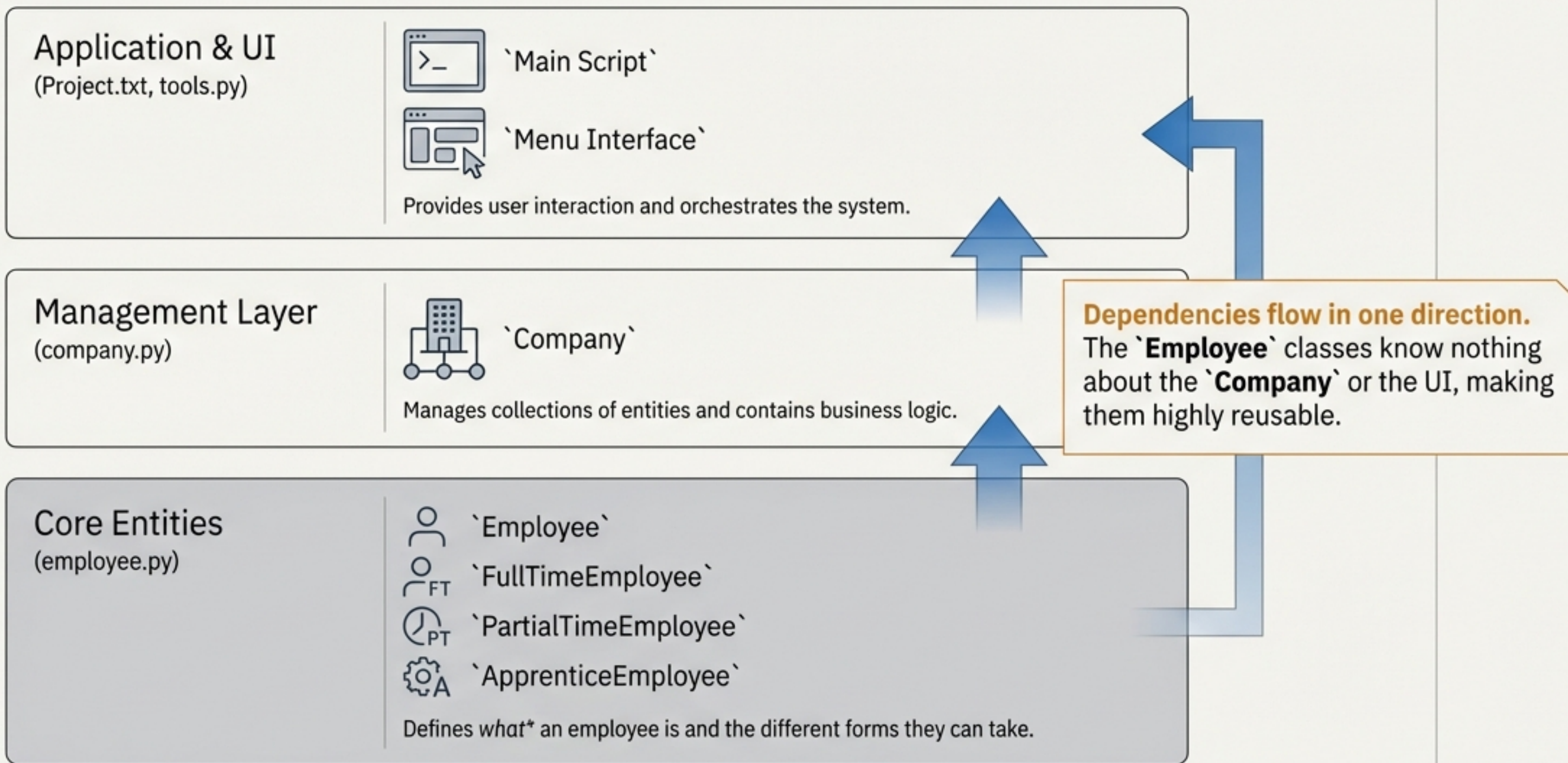


Anatomy of a Modern Payroll System

A guided tour of an object-oriented design in Python.



The system is composed of three core, decoupled layers.



The `Employee` class establishes a universal contract for all employee types.

`Employee`	
Attributes	
name	
id	
Methods	
display_info()	
calculate_salary()	A
get_nb_hours_per_month()	A

```
class Employee:
    # Base class for all employee types
    def __init__(self, name, id):
        self.name = name
        self.id = id

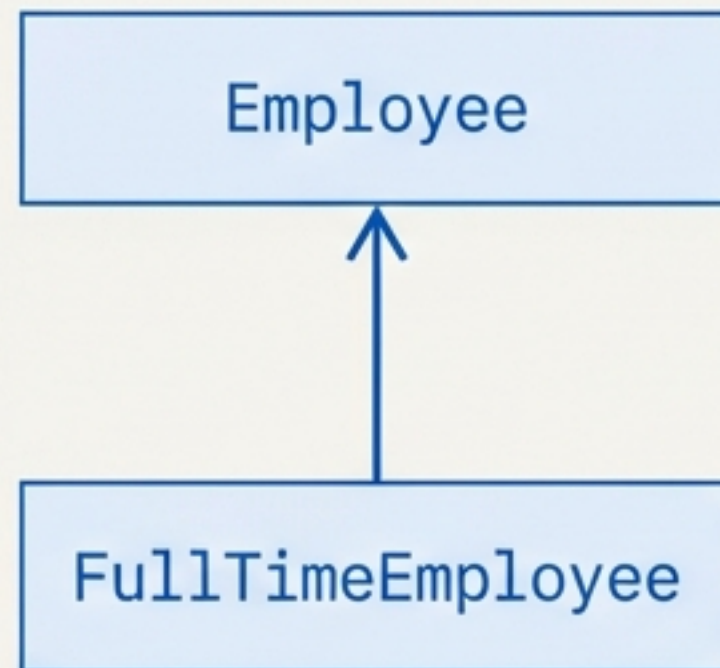
    def display_info(self):
        print(f"I am {self.name} and I have id {self.id}")

    def calculate_salary(self):
        raise NotImplementedError(...)

    def get_nb_hours_per_month(self):
        raise NotImplementedError(...)
```

→ This is a powerful design choice. It forces any developer creating a new employee type to provide a specific implementation for these methods, ensuring system-wide consistency and preventing runtime errors.

``FullTimeEmployee`` inherits the base contract and provides a concrete implementation.



```
class FullTimeEmployee(Employee):
    # works 156 hours per month
    def __init__(self, name, id, monthly_salary):
        super().__init__(name, id)
        self.monthly_salary = monthly_salary
        self.nb_hours_per_month = 156

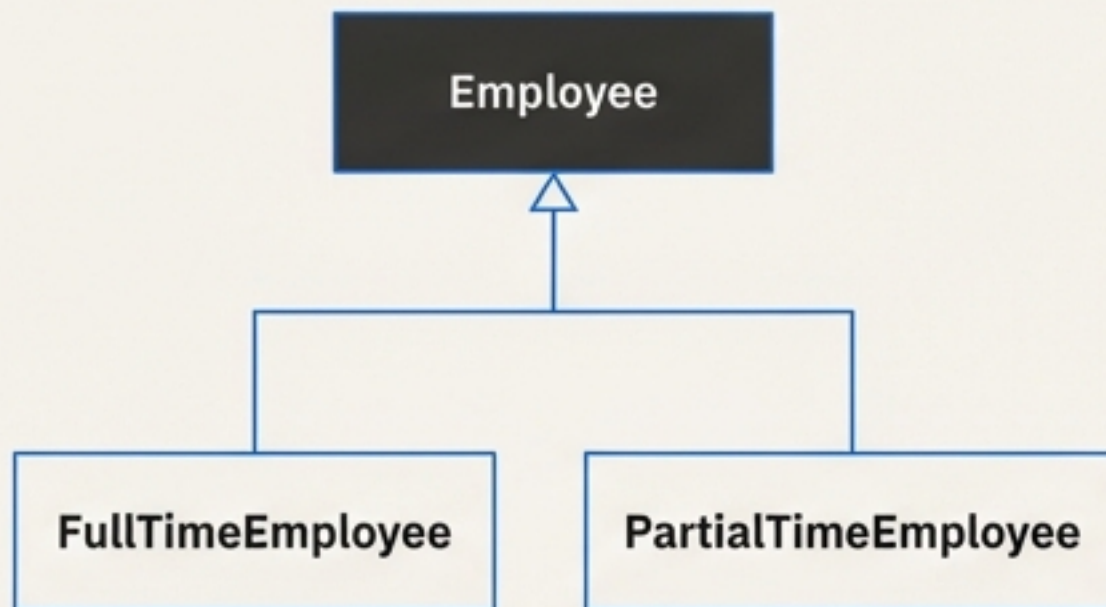
    def calculate_salary(self):
        self.salary = self.monthly_salary
        print(f"The salary of {self.name} ... is {self.salary:.2f} €")

    def get_nb_hours_per_month(self):
        return self.nb_hours_per_month
```

Reuses the initialization logic from the parent ``Employee`` class, avoiding code duplication.

Provides the required concrete implementation. The salary logic is simple: a fixed monthly amount.

`PartialTimeEmployee` demonstrates a different implementation of the same contract.



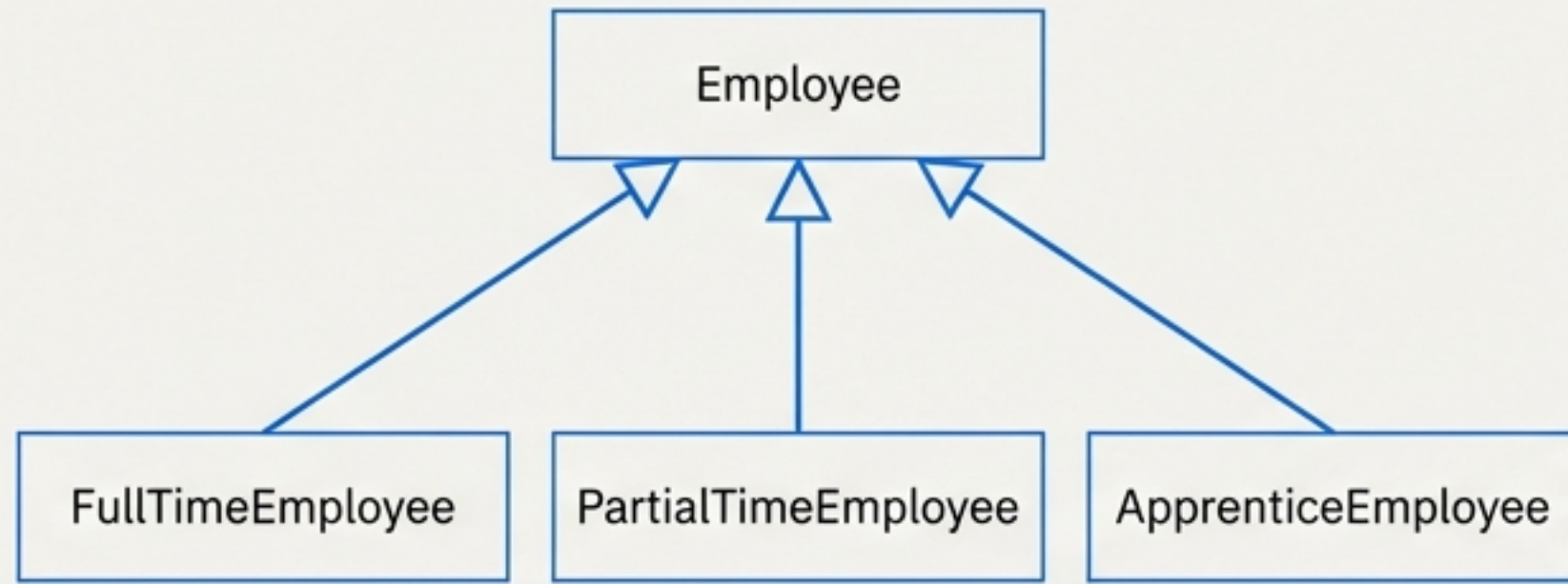
```
class PartialTimeEmployee(Employee):
    # variable hours and hourly pay
    def __init__(self, name, id, nb_hours_per_month, hourly_salary):
        super().__init__(name, id)
        self.nb_hours_per_month = nb_hours_per_month
        self.hourly_salary = hourly_salary

    def calculate_salary(self):
        self.salary = self.nb_hours_per_month * self.hourly_salary
        print(f"The salary of {self.name} .... is {self.salary:.2f} €")

    def get_nb_hours_per_month(self):
        return self.nb_hours_per_month
```

Note the different logic: `hours \* rate`. This showcases polymorphism. The 'what' (calculate salary) is the same, but the 'how' is specific to the class.

The `ApprenticeEmployee` class further illustrates the system's flexibility.

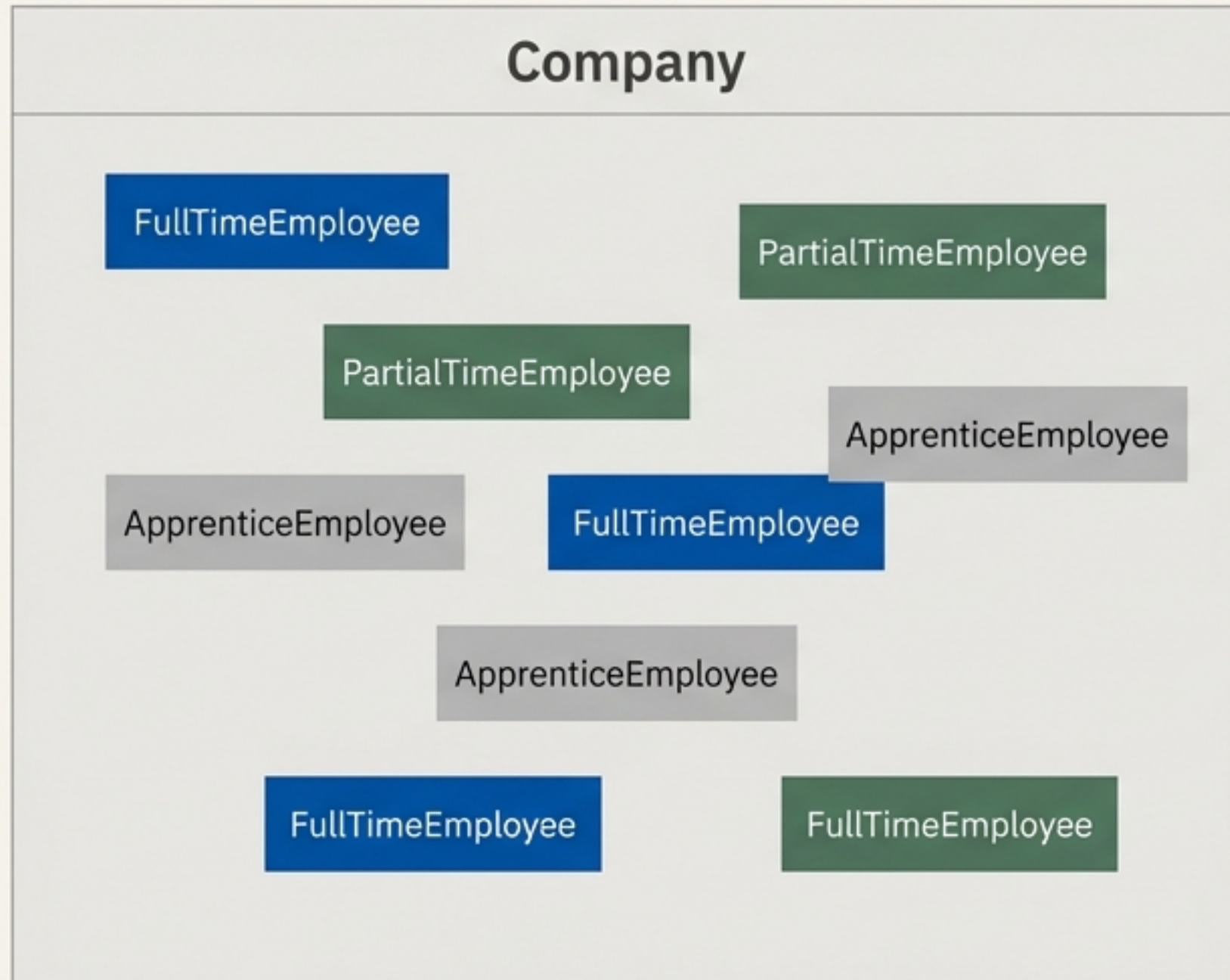


```
class ApprenticeEmployee(Employee):
    # earns 70% of minimum wage
    def __init__(self, name, id):
        super().__init__(name, id)
        self.minimum_wage = 1380
        self.nb_hours_per_month = 151
```

```
def calculate_salary(self):
    self.salary = self.minimum_wage * 0.7
    print(f"The salary of {self.name} ... is {self.salary:.2f} €")
```

A third distinct calculation logic, based on a fixed percentage of a base wage, fits seamlessly into the existing structure without requiring any changes to the base `Employee` contract.

The `Company` class aggregates and manages a collection of `Employee` objects.



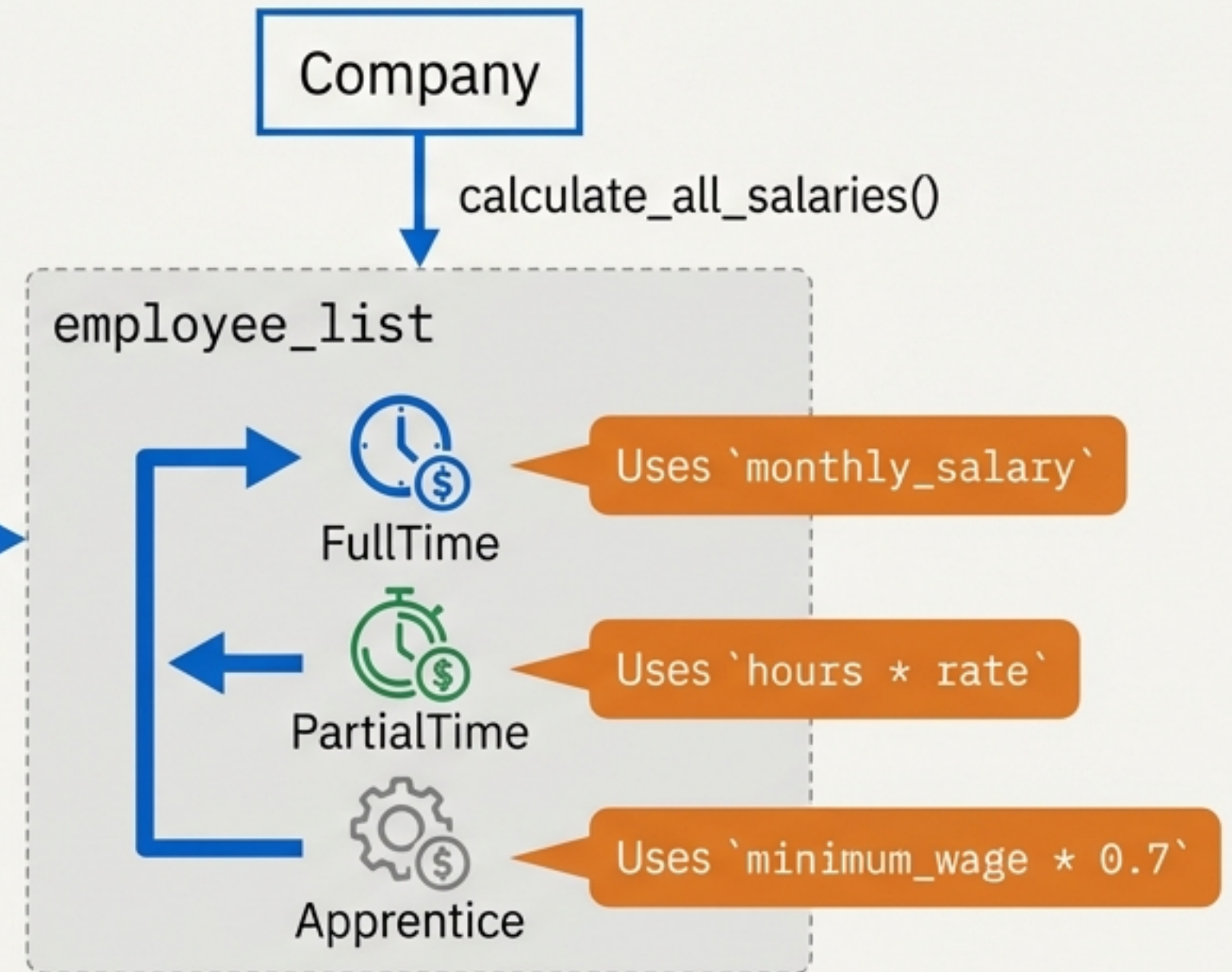
```
class Company:
    def __init__(self, name: str, employee_list=None):
        self.name = name
        self.employee_list = employee_list if
            employee_list is not None else []

    def add_employee(self, emp: Employee):
        self.employee_list.append(emp)
```

The `add_employee` method accepts any object that is a type of `Employee`. This is where the power of our inheritance structure pays off. The `Company` doesn't need to care about the specific type.

The company interacts with all employees uniformly, thanks to polymorphism.

```
def calculate_all_salaries(self):  
    print("\n***** Calculating employee salaries: *****")  
    for e in self.employee_list:  
        e.calculate_salary()  
    print("*****")
```



This is the magic of polymorphism. The **Company** class doesn't know or care *how* each salary is calculated. It simply trusts each **Employee** object to do the right thing when `calculate_salary()` is called. This dramatically simplifies the management code.

Helper methods provide a simplified interface for creating employees.

Without Helper Method

```
# User must import and instantiate the class
directly
from employee import FullTimeEmployee

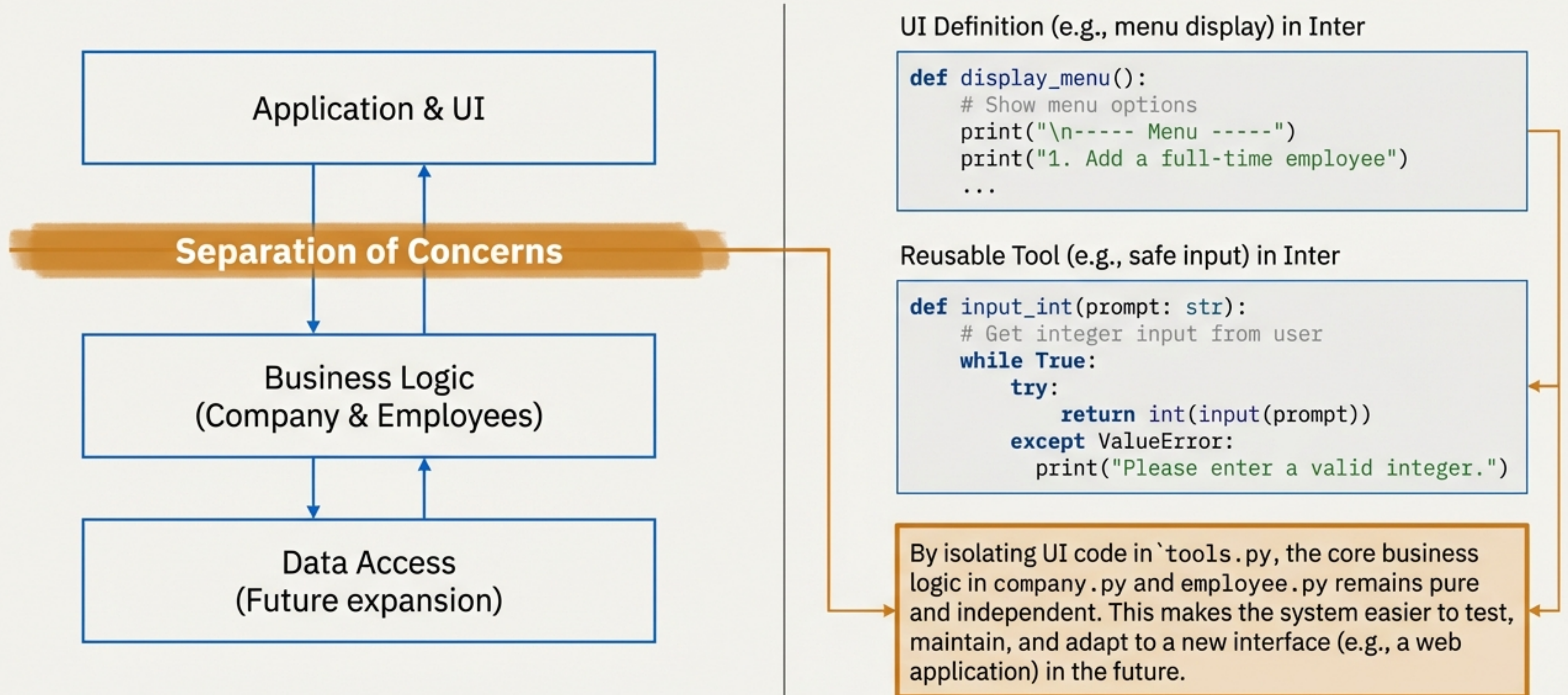
emp = FullTimeEmployee("Cocotte", 1402, 3000)
company.add_employee(emp)
```

With Helper Method

```
# Details of FullTimeEmployee are hidden
company.add_fulltime_employee(
    name="Cocotte",
    id=1402,
    monthly_salary=3000
)
```

These convenience methods (`add_fulltime_employee`, `add_PartialTime_employee`, etc.) are part of the `Company` class's public API. They encapsulate the object creation logic, making the main program cleaner and insulating it from future changes to the `Employee` subclasses.

User interaction logic is kept separate in a dedicated `tools` module.



The main script orchestrates the setup and initial state of the application.

1. For direct control, instantiate an object and pass it to `add_employee`.

```
# Create company
company = Company("Gil and Co.")

# Method 1: Direct objects
company.add_employee(PartialTimeEmployee("Jérôme", 72, ...)) # Highlight 1

# Method 2: Using convenience methods
company.add_fulltime_employee(name="Cocotte", id=1402, ...) # Highlight 2

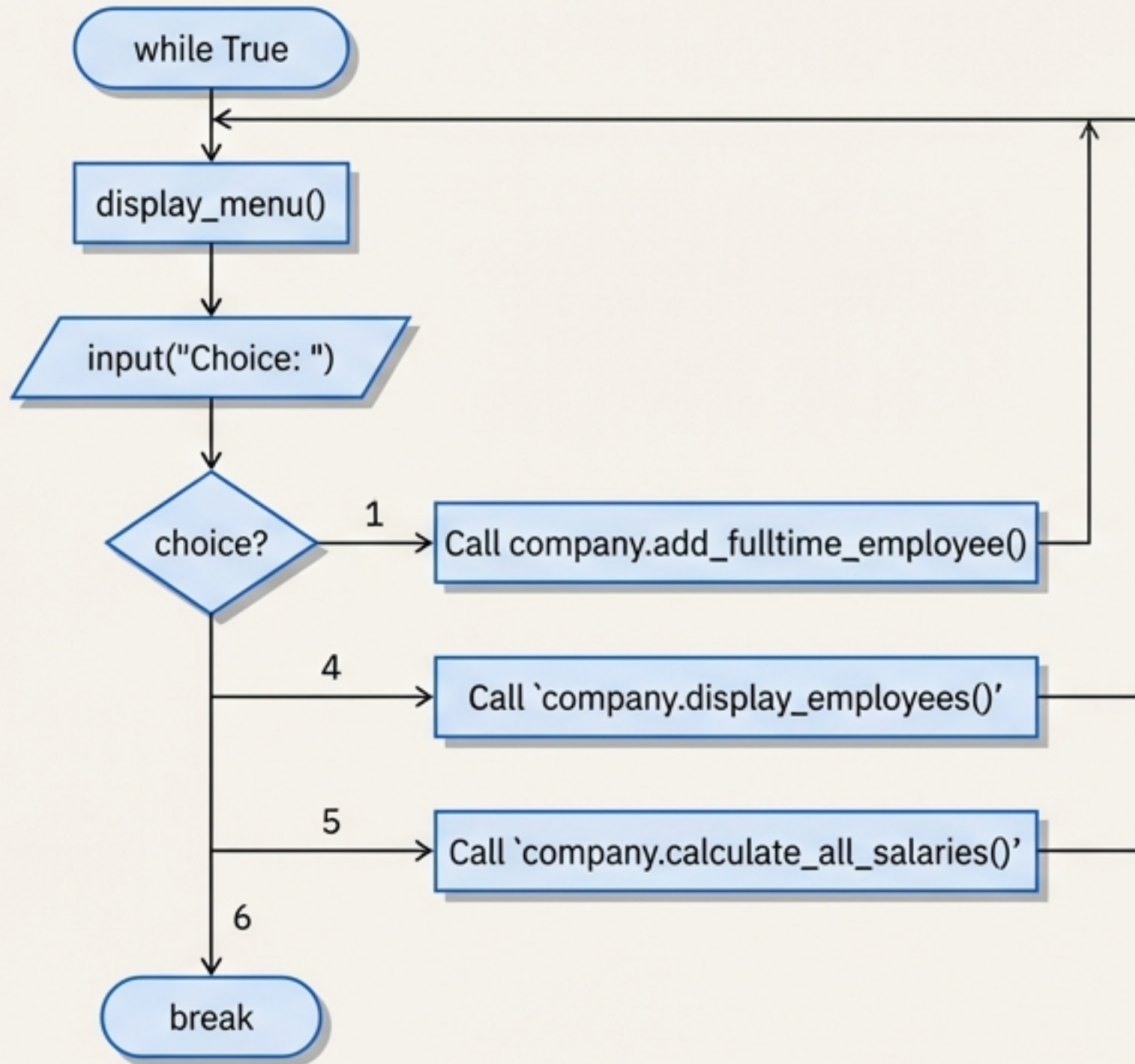
# Method 3: Add multiple employees at once
employee_list = []
employee_list.append(FullTimeEmployee("Titi", 15, 2000))
...
company.add_employees(employee_list) # Highlight 3
```

2. The cleaner, recommended approach using the helper API.

This demonstrates a well-designed, flexible API that accommodates multiple use cases for populating the 'Company' object.

3. For bulk operations, a list of objects can be added efficiently with `'add_employees'`.

An interactive menu loop drives the user experience.



```
def run_menu(company: Company):  
    while True:  
        display_menu()  
        choice = input("Choice: ").strip()  
        if choice == "1":  
            # ... get name, id, salary ...  
            company.add_fulltime_employee(...)  
        elif choice == "4":  
            company.display_employees()  
        elif choice == "5":  
            company.calculate_all_salaries()  
        elif choice == "6":  
            break
```

The `run_menu` function acts as the controller, translating user input into commands for the `Company` object, which contains the actual business logic.

This design elegantly demonstrates fundamental OOP principles.

Inheritance

The ``Employee`` class provides a common blueprint, which specialized classes like ``FullTimeEmployee`` and ``PartialTimeEmployee`` extend and build upon. This promotes code reuse.

Polymorphism

The ``Company`` class calls ``e.calculate_salary()`` on every employee in its list, and the correct, specific version of the method is executed for each object. This simplifies the management code.

Encapsulation

The ``Company`` class hides its internal ``employee_list`` and provides clean public methods like ``add_fulltime_employee()``. This protects the internal state and simplifies the interface.

Separation of Concerns

The UI logic in ``tools.py`` is completely distinct from the business logic in ``company.py`` and ``employee.py``. This makes the system modular, testable, and maintainable.

The result is a clean, maintainable, and extensible system.

```
\**** List of employees at Gil and Co.: ****  
I am Jérôme and I have id 72  
I am Toto and I have id 142  
...  
*****  
  
**** Calculating employee salaries: ****  
The salary of Jérôme (PartialTimeEmployee) is 2400.00 €  
...for 120.0 hours worked  
The salary of Toto (FullTimeEmployee) is 1500.00 €  
...for 156.0 hours worked  
...
```

Designed for the Future

The true strength of this design is how easily it accommodates change. What if we need to add a 'Contractor' with a daily rate? We would simply create a new `ContractorEmployee` class that inherits from `Employee` and implements `calculate_salary`. No changes would be needed in the `Company` class or the UI logic to manage and pay them. This is the definition of a robust architecture.

Thank You

Q & A

